



LIONSDEV

Curso de Programação

MÓDULO 3

Desenvolvimento de uma Calculadora



Professor Nicolas Cardoso Motta

Revisando o Módulo 2

1 2
3 4

Variáveis e tipos

`let` muda, `const` não.
Tudo que vem do `prompt` é texto:
converta com `Number()`.



Decisão e repetição

`if/else`, `switch`, `for` e `while` controlam o caminho do programa.



Arrays, objetos e funções

Listas, fichas com chaves e blocos com nome que recebem e devolvem valores.

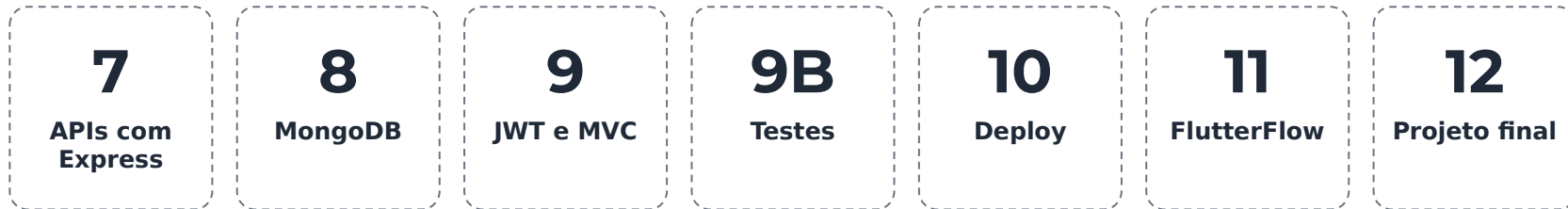
Pergunta para abrir: quanto dá `"2" + 3`? E `Number("2") + 3`?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



Você está aqui: **Módulo 3**, Calculadora e Clínica Médica



Objetivo

- Entender por que definimos requisitos e escopo antes de programar.
- Planejar o fluxo de execução de um programa interativo com menu.
- Validar os dados de entrada para deixar o programa robusto.
- Construir uma calculadora no terminal com funções, while e switch.
- Aplicar o mesmo processo num segundo projeto: a Clínica Médica.

**No Módulo 2 aprendemos as peças:
variáveis, condicionais, laços e
funções. Agora vamos juntar tudo no
nosso primeiro projeto de verdade.**



Qual calculadora?

"Fazer uma calculadora" pode significar muita coisa...

Três calculadoras, três projetos diferentes



Básica

Soma, subtração,
multiplicação e divisão,
com inteiros e
decimais.



Científica

Trigonometria, raízes,
potências, álgebra e
estatística.



Financeira

Juros simples e
compostos, valor
presente líquido e
investimentos.

Por que definir requisitos antes de codar?



Necessidades, funcionalidades e restrições

Deixa claro o que o sistema faz e o que ele não faz.



O produto certo

Garante que o resultado atenda às expectativas de quem vai usar.



Menos retrabalho

Evita atrasos e custos de refazer o que já estava pronto.

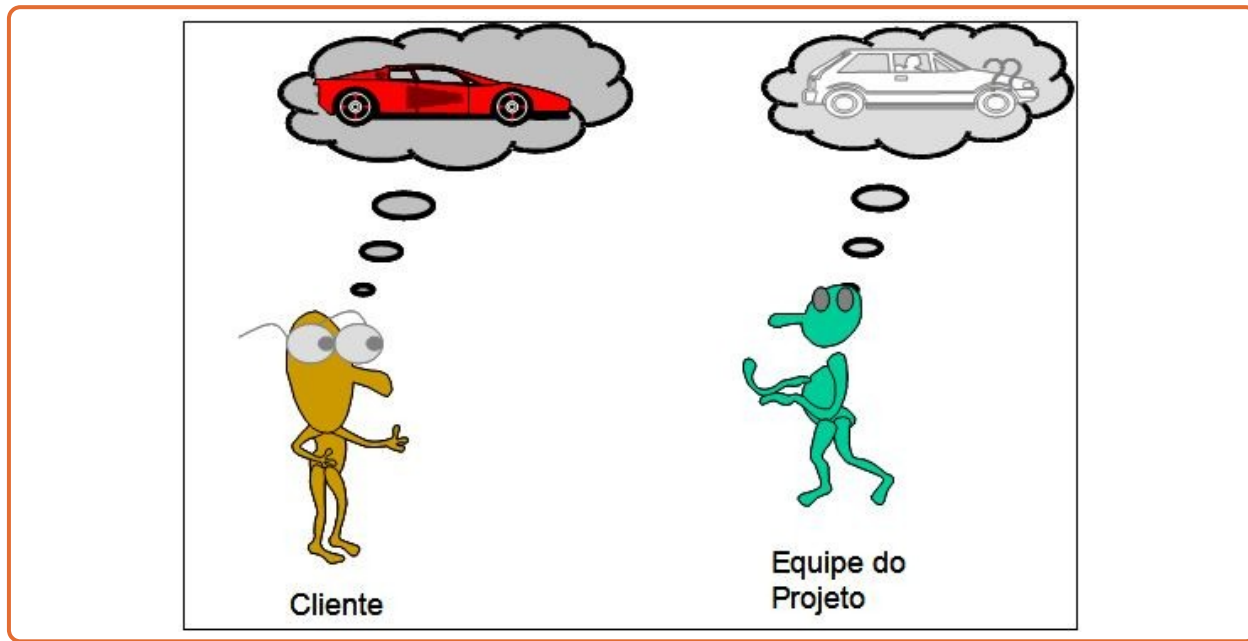


Você sabe quando terminou

E sabe exatamente o que precisa testar.

Imagine que você acabou de entrar num projeto cujo único requisito é: "vamos construir um carro". O que você imagina?

O que cada um entendeu



Sem requisitos claros, o cliente imagina uma Ferrari e a equipe entrega um Fiat Uno.

Escopo: qual calculadora vamos criar?



Dentro do escopo

- Adição, subtração, multiplicação e divisão
- Porcentagem sobre o resultado acumulado
- Ver o resultado atual
- Sair do programa



Fora do escopo

- Operações científicas (seno, log, potência)
- Histórico das operações em arquivo
- Interface gráfica

O que está fora do escopo pode virar uma melhoria depois. Primeiro, o combinado.

Critérios de aceite: quando está pronto?



O menu numérico aceita só opções válidas e avisa quando a opção é inválida.



Se o usuário digitar algo que não é número, o sistema avisa e o resultado não se perde.



O programa só termina quando o usuário escolhe sair.



Dá para ver o resultado acumulado sem fazer outra operação.



A divisão por zero mostra uma mensagem clara.

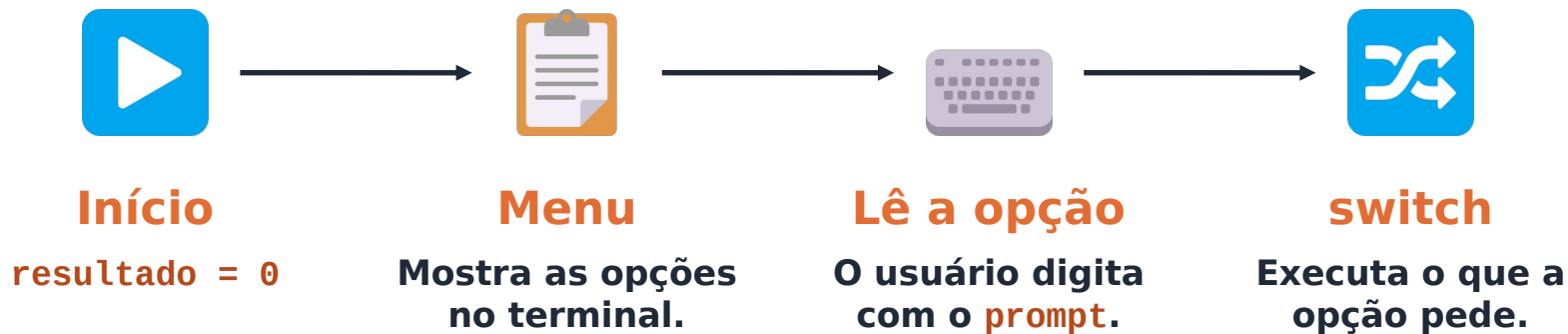


Cada operação fica na sua própria função, e o resultado acumula entre as operações.



Fluxo de Execução

O fluxo da calculadora



Depois de cada opção, o programa volta ao menu. Isso se repete até o usuário escolher sair.

O que cada opção faz?

1 a 5 Operações

Pede um número, valida e atualiza o resultado.

6 Ver resultado

Mostra o resultado atual, sem alterar nada.

7 Sair

Mostra "Encerrando..." e termina o laço.

outra Inválida

Avisa que a opção é inválida e volta ao menu.

Por que um while e um switch?



Interatividade

A cada volta do laço, o usuário vê o menu e decide o que fazer.



Menu

As opções numéricas (1 a 7) são tratadas no **switch**.



Múltiplas operações

O **while** mantém o programa vivo, operação após operação.



Resultado persistente

A variável **resultado** guarda o valor entre as operações, enquanto o programa roda.

Como o usuário vê a calculadora

```
Qual operação você deseja executar?  
[1] Somar [2] Subtrair [3] Multiplicar  
[4] Dividir [5] Porcentagem  
[6] Ver resultado [7] Sair  
Escolha uma opção: 1  
Digite o número para somar: 10  
Resultado: 10
```

menu → aparece de novo a cada volta do laço.

opção → chega como texto; convertamos com **Number()**.

resultado → começa em 0 e vai acumulando: $0 + 10 = 10$.



Validação dos Dados de Entrada

"A validação de dados é a base de uma aplicação confiável e segura. Validar entradas não é apenas uma boa prática: é uma **necessidade na programação."**



Podemos somar letras?

```
let resultado = 10  
let num = Number("dez")  
  
resultado = resultado + num  
console.log(resultado)
```

Pense antes de rodar: qual é a saída?



Cuidado: o NaN contamina tudo

Imprime **NaN**. E, a partir daí, **toda conta** com o resultado também dá NaN.

- **Number()** devolve **NaN** (Not a Number) quando o texto não é um número.
- O resultado acumulado se perde sem nenhum erro aparecer: é o "NaN silencioso".
- Por isso validamos o valor antes de usá-lo.

```
console.log(NaN + 5)    // NaN  
console.log(NaN * 0)    // NaN
```

Por que validar?



Prevenção de erros

O programa recebe os tipos e formatos que espera, e não tenta fazer contas com texto.



Robustez

Lida com entradas inesperadas com mensagens úteis, em vez de quebrar.



Manutenção

Quando algo dá errado, fica fácil saber se o problema foi a entrada.



Segurança

Dados não validados deixam o sistema vulnerável a erros e ataques.

typeof: qual é o tipo?

```
typeof 42           // "number"  
typeof "42"         // "string"  
typeof true         // "boolean"  
typeof NaN          // "number" (!)
```

typeof → devolve, em texto, o tipo do valor.

NaN → apesar do nome, é do tipo **number**. Por isso o **typeof** não basta para validar.

prompt → tudo que vem do **prompt** chega como **string**.

O padrão do projeto: Number + isNaN

```
let num = Number(prompt('Digite um número: '))

if (isNaN(num)) {
  console.log('Da próxima vez digite um número correto!')
} else {
  // usa o número com segurança
}
```

Três passos: ler com o prompt, converter com Number() e testar com isNaN().



Cuidado: Enter vazio vira zero

`Number("")` dá **0**, e o `isNaN` deixa passar.

- Na calculadora, um Enter sem querer soma 0, ou cai na divisão por zero.
- Na clínica, se a opção 0 for "Sair", um Enter vazio fecha o programa.
- Teste o texto antes de converter: `entrada.trim() === ""`.

```
Number("")           // 0
Number(" ")          // 0
isNaN(0)              // false: passou na validação!
```

Uma validação mais completa

```
let entrada = prompt('Digite um número: ').replace(',', ' .')
let num = Number(entrada)

if (entrada.trim() === '' || !Number.isFinite(num)) {
  console.log('Digite um número válido!')
} else {
  // usa o número com segurança
}
```

O `replace` aceita a vírgula decimal. O `trim()` barra o Enter vazio. E o `Number.isFinite` barra o NaN e também o Infinity: quem digita "Infinity" ou "1e999" passa pelo `isNaN`.

Nem tudo o isNaN resolve

```
// regra: o desconto precisa estar entre 0 e 100
if (desconto < 0 || desconto > 100) {
  console.log('O desconto deve estar entre 0 e 100')
}

// regra: não existe divisão por zero
if (num === 0) {
  console.log('Não é possível dividir por zero!')
}
```

Um valor pode ser um número válido e mesmo assim não fazer sentido. E $10 / 0$ dá Infinity, não NaN: o isNaN não pega esse caso.

Antes do código: pseudocódigo

```
resultado = 0
REPITA até a opção ser "sair":
    mostrar o menu
    ler a opção
    SE for uma operação:
        ler um número
        SE não for número: avisar
        SENÃO: calcular e guardar em resultado
    SE for "ver resultado": mostrar resultado
mostrar "Encerrando..."
```

Escrever a lógica em português antes, sem se preocupar com a sintaxe, deixa o código muito mais fácil de escrever depois.



Hora de Codar!

Vamos construir a calculadora juntos, passo a passo

Preparando o projeto

```
$ mkdir calculadora  
$ cd calculadora  
$ npm init -y  
$ npm install prompt-sync  
$ touch calculadora.js
```

npm init -y → cria o package.json.

prompt-sync → para ler o que o usuário digita.

"type": "module" → o código da aula usa **import**: adicione essa linha no package.json (lembra do Módulo 2?).

Passo 1: uma função para cada operação

```
import promptSync from 'prompt-sync'
const prompt = promptSync({ sigint: true })

function somar(num1, num2) { return num1 + num2 }
function subtrair(num1, num2) { return num1 - num2 }
function multiplicar(num1, num2) { return num1 * num2 }
function dividir(num1, num2) { return num1 / num2 }
function porcentagem(valor, percentual) { return (valor * percentual) / 100 }
```

Uma função por operação: cada uma devolve o resultado com return. Exemplo: porcentagem(200, 10) devolve 20 (10% de 200).

Passo 2: o menu e o laço

```
let op = 0
let resultado = 0
let num

while (op !== 7) {
  console.log('\nQual operação você deseja executar?')
  console.log('[1] Somar [2] Subtrair [3] Multiplicar')
  console.log('[4] Dividir [5] Porcentagem')
  console.log('[6] Ver resultado [7] Sair')

  op = Number(prompt('Escolha uma opção: '))

  switch (op) {
    // um case para cada opção (próximos passos)
  }
}
```

A variável resultado fica fora do laço para não zerar a cada volta. O while repete enquanto a opção for diferente de 7.



Cuidado: esqueceu o Number() no menu?

Sem o `Number()`, a opção é texto: `"1"` não bate com `case 1`, e o programa **nunca sai**.

- O `switch` compara com `===`: valor e tipo.
- E `"7" !== 7` é sempre verdadeiro: o `while` não termina, e tudo cai no default.

```
op = prompt('Escolha uma opção: ') // '7' (texto)
while (op !== 7) { ... }           // nunca termina
```

Passo 3: um case com validação

```
case 1:
  num = Number(prompt('Digite o número para somar: '))
  if (isNaN(num)) {
    console.log('Da próxima vez digite um número correto!')
  } else {
    resultado = somar(resultado, num)
    console.log('Resultado: ' + resultado)
  }
  break
```

Se não for número, avisa e não mexe no resultado. Os cases 2 e 3 seguem o mesmo modelo, com subtrair e multiplicar.



Cuidado: let dentro de um case

Todos os cases dividem o mesmo bloco: `let num` em dois cases dá `SyntaxError`.

- Por isso declaramos `let num` uma única vez, antes do laço.
- Outra saída: colocar chaves `{ }` em volta do código de cada case.

```
case 1:
  let num = Number(prompt('Somar: '))
  break
case 2:
  let num = Number(prompt('Subtrair: '))
  // SyntaxError: Identifier 'num' has already been declared
```

Passo 4: a divisão por zero

```
case 4:
  num = Number(prompt('Digite o número para dividir: '))
  if (isNaN(num)) {
    console.log('Da próxima vez digite um número correto!')
  } else if (num === 0) {
    console.log('Não é possível dividir por zero!')
  } else {
    resultado = dividir(resultado, num)
    console.log('Resultado: ' + resultado)
  }
  break
```

Uma segunda validação, com else if: primeiro testamos se é número, depois se é zero.

Passo 5: a porcentagem

```
case 5:
  num = Number(prompt('Digite a porcentagem: '))
  if (isNaN(num)) {
    console.log('Da próxima vez digite um número correto!')
  } else {
    let valor = porcentagem(resultado, num)
    console.log(`${num}% de ${resultado} = ${valor}`)
  }
  break
```

A porcentagem calcula e mostra o valor, sem alterar o resultado acumulado.

Passo 6: ver resultado, sair e opção inválida

```
case 6:  
  console.log('Resultado: ' + resultado)  
  break  
  
case 7:  
  console.log('Encerrando...')  
  break  
  
default:  
  console.log('Opção inválida, tente novamente!')  
  break
```

case 6 → só mostra o valor acumulado.

case 7 → o while testa **op !== 7** e o laço termina.

default → qualquer outra opção, inclusive texto (que vira NaN).

Testando os critérios de aceite

```
Escolha uma opção: 1
Digite o número para somar: 10
Resultado: 10
Escolha uma opção: 3
Digite o número para multiplicar: 3
Resultado: 30
Escolha uma opção: 4
Digite o número para dividir: 0
Não é possível dividir por zero!
Escolha uma opção: 1
Digite o número para somar: abc
Da próxima vez digite um número correto!
Escolha uma opção: 5
Digite a porcentagem: 10
10% de 30 = 3
Escolha uma opção: 7
Encerrando...
```

acumula → $10 \times 3 = 30$: o resultado passa de uma operação para outra.

protege → zero na divisão e "abc" geram aviso, e o resultado continua 30.

encerra → só quando o usuário escolhe 7.

teste você → para cada critério de aceite, tente fazer o programa falhar.

Refatorando: uma função para ler números

```
function lerNumero(mensagem) {  
  const entrada = prompt(mensagem).replace(',', ' ');  
  if (entrada.trim() === '') return NaN  
  return Number(entrada)  
}  
  
case 1:  
  num = lerNumero('Digite o número para somar: ' )  
  if (!Number.isFinite(num)) {  
    console.log('Da próxima vez digite um número correto!')  
  } else { ... }
```

A mesma validação se repetia em cinco cases. Uma função resolve de uma vez: é o princípio **DRY** (Don't Repeat Yourself). A calculadora do Módulo 5 no repositório usa essa versão.

Resultado com casas decimais

```
resultado = somar(0.1, 0.2)

console.log('Resultado: ' + resultado)
// Resultado: 0.30000000000000004

console.log('Resultado: ' + resultado.toFixed(2))
// Resultado: 0.30
```

Arredonde só na hora de mostrar. A variável resultado continua com o valor completo para as próximas contas.

Tente quebrar o seu programa

Digite...	O que testa	Esperado
Enter vazio	Entrada vazia	Aviso
abc	Texto no lugar de número	Aviso
7,5	Vírgula decimal	7.5 ou aviso claro
0 na divisão	Regra de negócio	Aviso
-5	Número negativo	Funciona
1e999	Número gigante	Aviso
9 no menu	Opção que não existe	Opção inválida

Testar é tentar fazer o programa falhar. Se ele sobreviver a tudo isso, está pronto.

Projeto Calculadora

Escopo mínimo

O projeto só está completo quando todos os itens funcionam:

1. Menu numérico que avisa quando a opção é inválida.
2. Entrada que não é número gera aviso e não altera o resultado.
3. O laço só termina quando o usuário escolhe sair.
4. Existe a opção de ver o resultado acumulado.
5. A divisão por zero mostra uma mensagem clara.
6. Cada operação está na sua própria função, e o resultado acumula entre as operações.



Dica: o código feito em aula está no repositório, em `modulo03/exercicios_slides/calculadora.js`.

Indo além (opcional)



Pitágoras

Hipotenusa a partir de dois catetos:

`Math.sqrt(a * a + b * b).`



Potência e raiz

Com `Math.pow` e `Math.sqrt`.



Limite e derivada

Aproximações numéricas, com valores cada vez mais próximos do ponto.



Ideias suas

Média, fatorial, conversão de unidades...

Só depois que o escopo mínimo estiver funcionando. Cada extra segue o padrão: função, validação e um case no switch.

Lista de exercícios



Parte 0: aquecimento

Funções, validações e switch em peças soltas.



Parte 1: complete o código

Validador de número e operações com switch.



Parte 2: ache o bug

A divisão que devolve Infinity.



Desafio final

Juntar todos os pedaços numa calculadora completa.

No repositório: `modulo03/lista_de_exercicios/codigo_calculadora.md`



Projeto 2: Clínica Médica

O mesmo processo, agora com arrays e objetos

O que o sistema faz?



Agendamento de consultas

Um sistema para a clínica, rodando no terminal com Node.js.



Menu de ações

Adicionar, listar, atualizar e cancelar consultas, e sair.



Objetos dentro de um array

Cada consulta é um objeto; todas ficam guardadas num array.



Validação

Número de consulta que não existe gera aviso, sem alterar nada.

Escopo da clínica



Dentro do escopo

- Adicionar consulta (paciente, médico, data e hora)
- Listar todas as consultas
- Atualizar uma consulta
- Cancelar uma consulta
- Sair do programa



Fora do escopo

- Cadastro separado de médicos e pacientes
- Salvar em arquivo ou banco de dados
- Interface gráfica

Cada consulta é um objeto

```
let consultas = []

consultas.push({
  paciente: 'Maria',
  medico: 'Dr. Carlos',
  data: '2026-10-05',
  hora: '14:30'
})
```

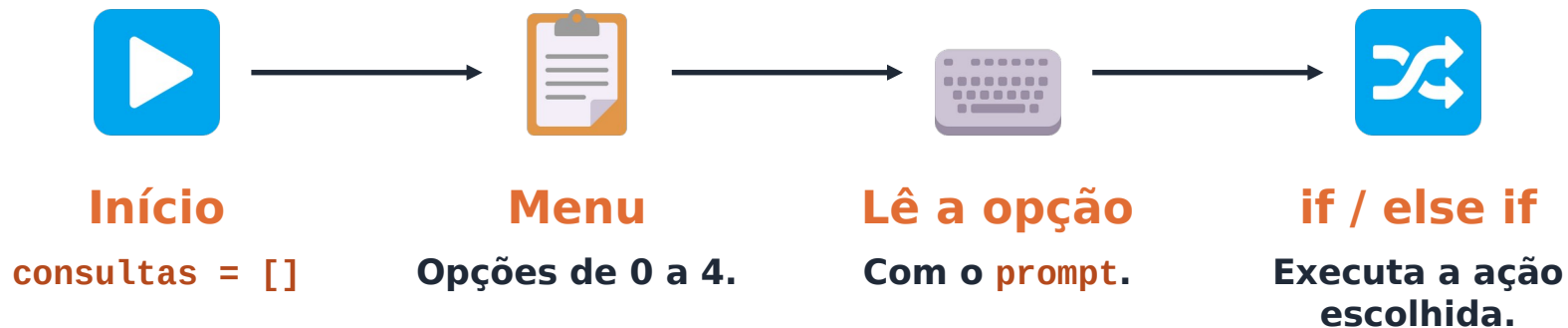
consultas → o array que guarda todas as consultas.

{ ... } → cada consulta tem quatro campos: paciente, médico, data e hora.

push → adiciona a nova consulta no fim do array.

memória → fechou o programa, os dados somem. Banco de dados fica para o Módulo 8.

O fluxo da clínica



Depois de cada ação, volta ao menu. A opção 0 encerra o programa.

O laço da clínica: do...while

```
let op

do {
  console.log('\n[1] Adicionar [2] Listar [3] Atualizar')
  console.log('[4] Cancelar [0] Sair')
  op = Number(prompt('Escolha uma opção: '))

  if (op === 1) {
    // adicionar
  } else if (op === 2) {
    // listar
  } // ...
} while (op !== 0)
```

Com `let op = 0` e `while (op !== 0)`, o menu nem apareceria. O `do...while` mostra o menu pelo menos uma vez.

O que cada opção faz?

1 Adicionar

Lê os quatro campos, monta o objeto e usa **push**.

2 Listar

Mostra cada consulta com um número na frente: 1, 2, 3...

3 Atualizar

O usuário escolhe a consulta pelo número da listagem.

4 Cancelar

Remove a consulta escolhida com **splice**.

0 Sair

Encerra o laço e o programa.

Listando as consultas

```
if (consultas.length === 0) {  
  console.log('Nenhuma consulta agendada.')  
} else {  
  for (let i = 0; i < consultas.length; i++) {  
    let c = consultas[i]  
    console.log(`${i + 1}. ${c.paciente} - ${c.medico}`)  
    console.log(`    ${c.data} - ${c.hora}`)  
  }  
}
```

O `i + 1` faz a listagem começar em 1, mesmo com o array começando em 0.

Número da consulta x posição no array

O usuário vê a listagem

1. Maria - Dr. Carlos
2026-10-05 - 14:30
2. João - Dra. Ana
2026-10-06 - 09:00

No código

```
let num = Number(prompt('Número: '))  
let posicao = num - 1  
// 1 vira 0, 2 vira 1...  
  
consultas.splice(posicao, 1)
```

A listagem começa em **1**, mas o array começa em **0**: subtraia 1 do número digitado.

Validando o número da consulta

```
let num = Number(prompt('Digite o número da consulta: '))
let posicao = num - 1

if (isNaN(num) || posicao < 0 || posicao >= consultas.length) {
  console.log('Consulta não encontrada.')
} else {
  // atualiza ou cancela a consulta
}
```

Basta uma condição ser verdadeira para a consulta não existir: não é número, é menor que 1 ou é maior que a quantidade de consultas.

Atualizando uma consulta

```
let consulta = consultas[posicao]

let novaData = prompt(`Nova data (${consulta.data}): `)
let novaHora = prompt(`Nova hora (${consulta.hora}): `)

consulta.data = novaData || consulta.data
consulta.hora = novaHora || consulta.hora
```

O objeto é uma referência: alterar `consulta.data` já altera o item dentro do array. E o `||` mantém o valor antigo quando o usuário só aperta Enter (texto vazio é falsy).

Projeto Clínica Médica

Escopo mínimo

O projeto só está completo quando todos os itens funcionam:

1. Menu numérico que avisa quando a opção é inválida; o laço só termina ao escolher sair.
2. Cada consulta é um objeto com paciente, medico, data e hora.
3. A listagem mostra as consultas numeradas a partir de 1.
4. Sem consultas, a listagem avisa "Nenhuma consulta agendada.".
5. Atualizar e cancelar só funcionam com um número que existe; senão, o sistema avisa.
6. Depois de cancelar, a consulta não aparece mais na listagem.



Dica: o enunciado completo está no repositório, em `modulo03/conteudo/projeto_clinica_medica.md`.

Indo além (opcional)



Campos obrigatórios

Não aceitar paciente ou médico em branco.



Horário ocupado

Impedir duas consultas com o mesmo médico, na mesma data e hora.



Buscas

Consultas de um paciente ou de um dia específico.



Contador

Quantas consultas cada médico tem.

Outras ideias: especialidade do médico, telefone do paciente, status da consulta (agendada, realizada, cancelada).

Próximos passos



Módulo 4: Git e GitHub

Versionar os projetos e colaborar com a equipe.



Módulo 5: import e export

A calculadora dividida em vários arquivos, uma operação em cada.



Módulo 6: organização de projetos

A clínica em vários arquivos, com médicos e pacientes cadastrados.

Material do módulo



Aponte a câmera do celular

`github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo03`



conteudo/
teoria comentada e o cheat sheet do módulo



exercicios_slides/
os exercícios feitos em aula



lista_de_exercicios/
listas para praticar em casa



Fim!

Bora codar: é programando que se aprende a programar.